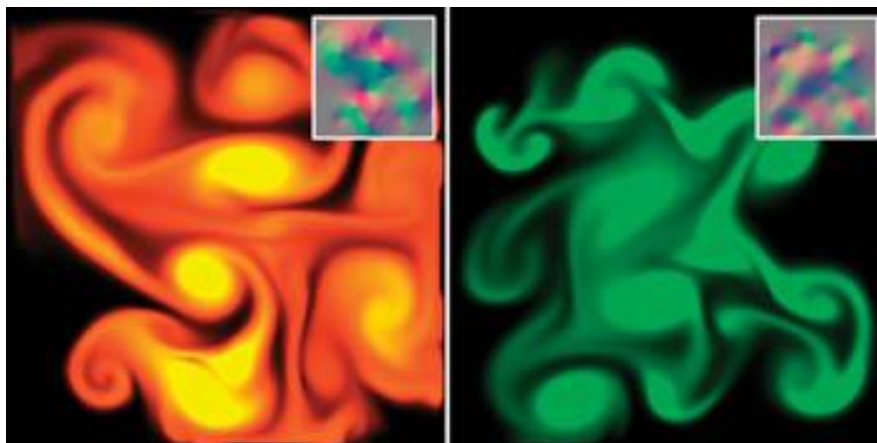


GPU에서의 빠른 유체 역학 시뮬레이션

Fast Fluid Dynamics Simulation on the GPU

Mark J. Harris

노스캐롤라이나 대학교 채플힐



한국어 번역 · 원문 수식·그림 보존

사용자 제공 GPU_gems_1.pdf / PDF 336-352쪽

목차

38.1 소개	4
38.1.1 우리의 목표	4
38.1.2 독자에게 기대하는 배경지식	4
38.1.3 접근 방법	5
38.2 수학적 배경	6
38.2.1 비압축성 흐름의 나비에-스토크스 방정식	7
38.2.2 나비에-스토크스 방정식의 각 항	8
38.2.3 벡터 미적분의 간단한 복습	8
38.2.4 나비에-스토크스 방정식 풀기	9
38.3 구현	17
38.3.1 CPU와 GPU의 대응 관계	17
38.3.2 슬랩 연산(Slab Operations)	19
38.3.3 프래그먼트 프로그램 구현	19
38.4 응용	25
38.4.1 액체와 기체 시뮬레이션	25
38.4.2 부력과 대류	25
38.5 확장	28
38.5.1 와도 보존(Vorticity Confinement)	28
38.5.2 3차원	29
38.5.3 엇갈린 격자(Staggered Grid)	29
38.5.4 임의의 경계	29
38.5.5 자유 표면 흐름	29
38.6 결론	30
38.7 참고문헌	31

번역 범위와 보존 원칙

사용자가 제공한 GPU_gems_1.pdf의 PDF 336-352쪽에 실린 38장 전체(38.1-38.7)를 번역했다. 본문의 순서, 논증, 수치, 수식 번호, 그림 7개, 표 1개와 코드 예제 5개를 유지했다. 수식·그림·표는 제공된 PDF에서 직접 추출한 원본 이미지를 사용하며, 코드의 실행 구문은 유지하고 주석만 번역했다. 그림 안의 영문 표기는 원본을 보존하고 필요한 한국어 풀이를 캡션에 덧붙였다. Cg, fragment program, 텍스처 제약과 성능 수치는 원문 집필 당시의 설명이다. Vulkan용으로 개작하지 않았다. 명백한 표기 불일치나 PDF의 문자 손실은 별도의 '옴긴이 주'로 구분했다.

출처: 사용자 제공 PDF 336쪽부터. PDF 쪽 번호는 파일 내의 1부터 시작하는 페이지 번호다.

이 장에서는 전적으로 GPU에서 실행되는 빠르고 안정적인 유체 시뮬레이션 방법을 설명한다. 유체 역학과 관련 수학을 소개하고, GPU에서 시뮬레이션을 수행하는 기법을 상세히 다룬다. 이 장을 읽고 나면 유체 역학의 기초를 이해하고 GPU를 사용해 유체를 시뮬레이션하는 방법을 알게 될 것이다. 책에 동봉된 소스 코드는 이 장에서 설명하는 기법을 시연한다.

38.1 소개

유체는 어디에나 있다. 강둑 사이로 흐르는 물, 타오르는 담배 끝에서 말려 올라가는 연기, 찻주전자에서 뿜어져 나오는 증기, 구름으로 뭉치는 수증기, 통 안에서 섞이는 페인트가 모두 그 예다. 이 모든 현상의 바탕에는 유체의 흐름이 있다. 모두 대화형 그래픽 애플리케이션에서 사실적으로 표현하고 싶은 현상들이다. 그림 38-1은 책에 제공된 소스 코드로 시뮬레이션한 유체의 예를 보여준다.

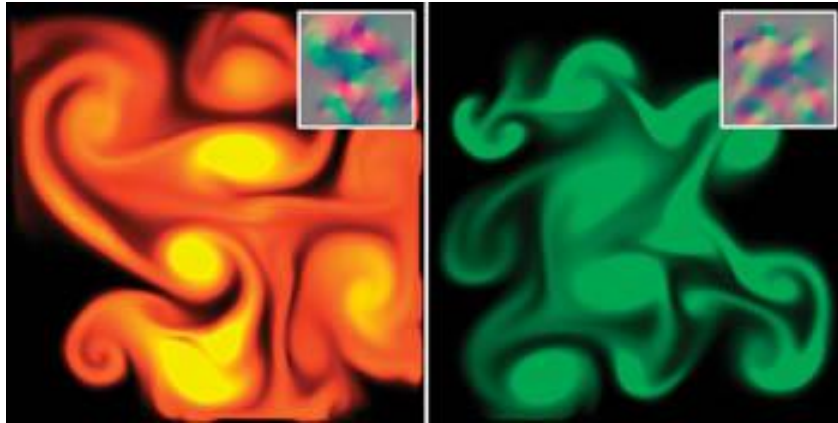


그림 38-1. 소용돌이치는 유체에 실려 이동하는 색색의 '염료'.

유체 시뮬레이션은 다양한 자연 현상을 시뮬레이션하는 토대가 되는 유용한 구성 요소다. 그래픽 하드웨어가 제공하는 높은 병렬성 덕분에, 여기서 설명하는 시뮬레이션은 CPU보다 GPU에서 훨씬 빠르게 실행된다. NVIDIA GeForce FX를 사용했을 때 동등한 CPU 시뮬레이션보다 최대 여섯 배의 속도 향상을 얻었다.

옮긴이 주 — 위 성능 수치는 원문 저자가 당시 하드웨어에서 보고한 결과다. 이 번역 작업에서 측정한 결과나 현재 CPU-GPU의 비교 수치가 아니다.

38.1.1 우리의 목표

우리의 목표는 새로운 요령 하나를 가르치는 데 그치지 않고, 강력한 도구를 익히도록 돕는 것이다. 유체 역학은 더 복잡한 시뮬레이션의 구성 요소로 매우 유용하므로, 이를 블랙박스로 취급하는 것은 잘못이다. 유체의 기초 물리와 수학을 이해하지 못하면 이 장의 알고리즘을 사용하거나 확장하기가 매우 어려울 것이다. 그래서 수학적 설명을 아끼지 않았다. 그 결과 이 장에는 얼핏 부담스럽게 보일 수 있는 수식이 많이 등장한다. 가능한 곳에서는 명확한 설명을 제공하고, 수학과 구현이 어떻게 연결되는지 보여준다.

38.1.2 독자에게 기대하는 배경지식

독자는 미분방정식에 대한 기초적인 이해를 포함해 대학 수준의 미적분학 배경을 갖추고 있다고 가정한다. 벡터 미적분의 원리를 알고 있으면 도움이 되지만 필수는 아니다. 필요한 내용은 다시 살펴볼 것이다. 도함수를 유한차분으로 근사해 본 경험도 도움이 된다. 포물선 운동이나 강체 역학 등 어떤 형태로든 물리 시뮬레이션을 구현해 보았다면, 여기서 사용하는 여러 개념이 익숙할 것이다.

38.1.3 접근 방법

원문 PDF 336-337쪽

여기서 설명하는 기법은 Stam(1999)의 ‘안정 유체(stable fluids)’ 방법에 기반한다. 다만 Stam의 시뮬레이션은 CPU로 구현되었지만, 우리는 그래픽 하드웨어에 구현한다. GPU가 유체 시뮬레이션에 필요한 종류의 계산에 잘 맞기 때문이다. 이 시뮬레이션은 셀로 구성된 격자에서 수행된다. 프로그래밍 가능한 GPU는 픽셀 계산에 최적화되어 있으며, 픽셀 역시 셀 격자로 볼 수 있다. GPU는 여러 정점과 픽셀을 동시에 처리하는 병렬성을 통해 높은 성능을 낸다. 또한 한 사이클에 여러 텍스처 조회를 수행하도록 최적화되어 있다. 시뮬레이션 격자를 텍스처에 저장하므로, 이 속도와 병렬성은 바로 우리가 필요로 하는 특성이다.

이 장에서 유체 역학의 모든 것을 가르칠 수는 없다. 다룰 수 있는 시뮬레이션 개념의 범위에는 한계가 있다. 여기서는 2차원 직사각형 영역을 채우는 연속적인 유체 체적의 시뮬레이션으로 범위를 제한한다. 또한 출렁이는 물과 공기의 접면처럼 서로 다른 유체 사이에 생기는 자유 표면(free surface) 경계는 시뮬레이션하지 않는다. 이 기본 기법에는 다양한 확장이 가능하다. 장의 끝에서 몇 가지를 소개하고 추가로 읽을 자료를 안내한다.

장 전체에서 수학 표기법을 일관되게 사용한다. 수식에서 압력 p 와 같은 스칼라량을 나타내는 변수는 이탤릭체로, 속도 u 와 같은 벡터량은 굵은 글씨로 표시한다. 이 장에 등장하는 모든 벡터는 2차원이라고 가정한다.

38.2절에서는 유체 흐름을 지배하는 방정식을 설명하고, 기본적인 벡터 미적분 개념과 표기법을 복습한 다음, 방정식의 해를 구하는 접근 방법을 살펴본다. 38.3절은 GPU에서의 유체 시뮬레이션 구현을 다룬다. 38.4절에서는 응용 사례를, 38.5절에서는 확장 방법을 소개하며, 38.6절에서 장을 마무리한다.

38.2 수학적 배경

유체의 거동을 시뮬레이션하려면 임의의 시점에서 유체의 상태를 수학적으로 표현할 수 있어야 한다. 가장 중요한 양은 유체의 속도다. 속도가 유체 자체와 그 안에 있는 것들의 움직임을 결정하기 때문이다. 유체의 속도는 시간과 공간에 따라 달라지므로 이를 벡터장(vector field)으로 표현한다.

벡터장은 데카르트 격자와 같이 매개변수로 표현한 공간에 벡터값 함수를 대응시키는 것이다. 다른 공간 매개화도 가능하지만 이 장에서는 2차원 데카르트 격자를 가정한다. 유체의 속도 벡터장은 각 위치 $x = (x, y)$ 마다 시간 t 에서의 속도 $u(x, t) = (u(x, t), v(x, t), w(x, t))$ 가 대응하도록 정의한다. 그림 38-2가 이를 보여준다.

움킨이 주 — 위 속도 정의에는 원문대로 세 성분 u, v, w 를 남겼다. 다만 이 장은 2차원을 가정하고 이후에는 $u = (u, v)$ 를 사용한다. 3차원 속도는 38.5.2절에서 다시 다룬다.

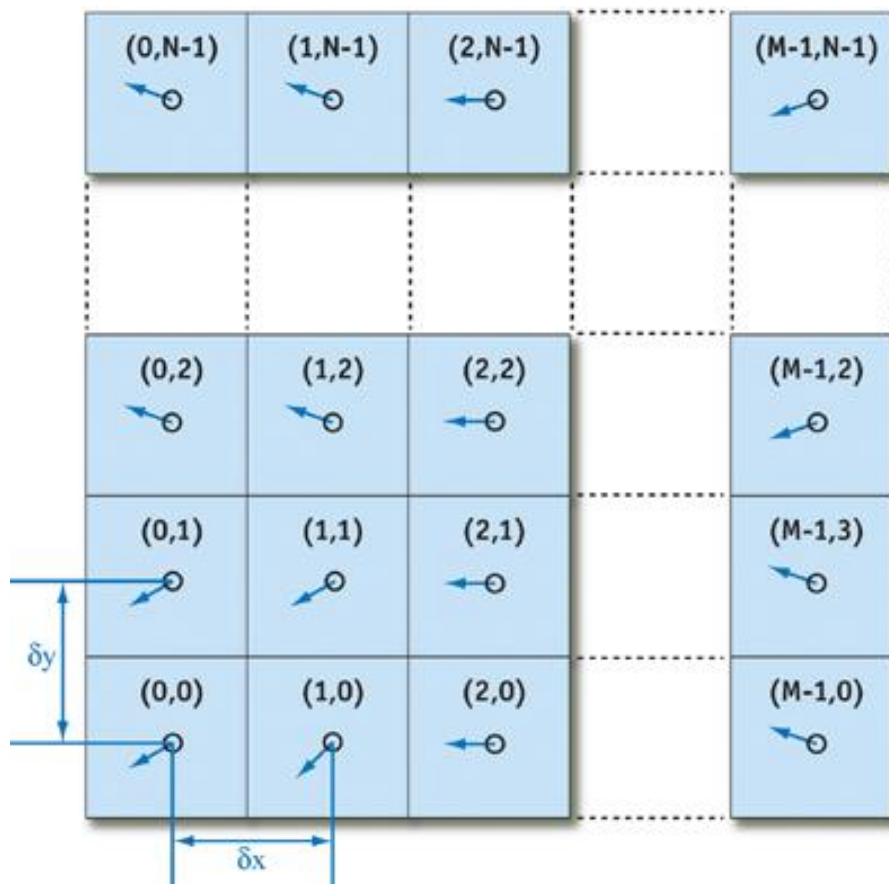


그림 38-2. 유체의 속도 격자. 셀 간격 $\delta x, \delta y$ 와 셀별 속도 성분을 나타낸다.

유체 시뮬레이션의 핵심은 시간을 한 단계씩 진행하면서 각 단계의 현재 속도장을 올바르게 결정하는 것이다. 다양한 힘을 받는 속도장이 시간에 따라 어떻게 변하는지를 기술하는 방정식들을 풀면 이를 수행할 수 있다. 속도장을 얻고 나면 이를 이용해 물체, 연기 밀도, 그리고 애플리케이션에서 표시할 수 있는 다른 양들을 이동시키는 흥미로운 일을 할 수 있다.

38.2.1 비압축성 흐름의 나비에-스토크스 방정식

원문 PDF 338쪽

물리학에서는 복잡한 현상을 모델링할 때 문제를 단순화하는 가정을 흔히 사용한다. 유체 시뮬레이션도 예외는 아니다. 여기서는 유체가 비압축성(incompressible)이고 균질(homogeneous)하다고 가정한다.

유체의 임의의 부분 영역이 차지하는 부피가 시간에 따라 일정하면 비압축성 유체라고 한다. 밀도 ρ 가 공간 전체에서 일정하면 균질한 유체라고 한다. 비압축성과 균질성을 함께 가정하면 밀도가 시간과 공간 모두에 대해 일정해진다. 이 가정은 유체 역학에서 흔히 사용되며, 그 결과로 얻는 수학을 물이나 공기 같은 실제 유체의 시뮬레이션에 적용하는 데에도 지장을 주지 않는다.

공간 좌표 $\mathbf{x} = (x, y)$ 와 시간 변수 t 를 사용하는 규칙적인 데카르트 격자에서 유체 역학을 시뮬레이션한다. 앞서 설명한 속도장 $\mathbf{u}(x, t)$ 와 스칼라 압력장 $p(x, t)$ 로 유체를 표현한다. 두 장 모두 공간과 시간에 따라 변한다. 초기 시각 $t = 0$ 의 속도와 압력을 알고 있다면, 시간에 따른 유체의 상태는 비압축성 흐름의 나비에-스토크스 방정식으로 기술할 수 있다.

$$\frac{\partial \mathbf{u}}{\partial t} = -(\mathbf{u} \cdot \nabla) \mathbf{u} - \frac{1}{\rho} \nabla p + \nu \nabla^2 \mathbf{u} + \mathbf{F},$$

식 1

$$\nabla \cdot \mathbf{u} = 0,$$

식 2

여기서 ρ 는 일정한 유체 밀도, ν 는 동점성 계수(kinematic viscosity), $\mathbf{F} = (f_x, f_y)$ 는 유체에 작용하는 외력을 나타낸다. $\mathbf{u}$ 가 벡터량이므로 식 1은 실제로 다음 두 식이다.

$$\frac{\partial u}{\partial t} = -(\mathbf{u} \cdot \nabla) u - \frac{1}{\rho} \nabla p + \nu \nabla^2 u + f_x,$$

식 1의 수평 속도 성분

$$\frac{\partial v}{\partial t} = -(\mathbf{u} \cdot \nabla) v - \frac{1}{\rho} \nabla p + \nu \nabla^2 v + f_y.$$

식 1의 수직 속도 성분

따라서 미지수는 u, v, p 의 세 개이며, 방정식 역시 세 개다.

나비에-스토크스 방정식은 처음에는 부담스러워 보일 수 있다. 그러나 다른 복잡한 개념처럼 단순한 조각으로 나누면 더 잘 이해할 수 있다. 식에 등장하는 개별 수학 연산이 아직 이해되지 않아도 걱정하지 말자. 먼저 유체의 흐름에 영향을 주는 요인들을 이해해 보겠다. 식 1의 우변에 있는 네 항은 모두 가속도다. 각각을 차례로 살펴보자.

38.2.2 나비에-스토크스 방정식의 각 항

이류(Advection)

유체의 속도에 의해 물체, 밀도, 그 밖의 양들이 흐름을 따라 운반된다. 움직이는 유체에 염료를 뿌리는 모습을 생각해 보자. 염료는 유체의 속도장을 따라 운반된다. 이를 이류된다고 한다. 실제로 유체의 속도는 염료를 운반하는 것과 마찬가지로 자기 자신도 운반한다. 식 1 우변의 첫 번째 항은 속도장의 이러한 자기 이류(self-advection)를 나타내며, 이류항이라고 부른다.

압력(Pressure)

유체 분자들은 서로의 주위를 이동할 수 있기 때문에 눌리거나 출렁이는 경향을 보인다. 유체에 힘을 가해도 그 힘이 유체 전체에 즉시 전달되지는 않는다. 힘이 가해진 곳에 가까운 분자들이 더 멀리 있는 분자들을 밀면서 압력이 쌓인다. 압력은 단위 면적당 힘이므로 유체 내부의 압력은 자연스럽게 가속도를 발생시킨다. 뉴턴의 제2법칙 $F = ma$ 를 떠올려 보자. 두 번째 항인 압력항은 이 가속도를 나타낸다.

확산(Diffusion)

실제 유체를 경험해 보면 어떤 유체는 다른 유체보다 더 걸쭉하다는 것을 알 수 있다. 예를 들어 당밀과 메이플 시럽은 천천히 흐르지만 소독용 알코올은 빠르게 흐른다. 걸쭉한 유체는 점성(viscosity)이 높다고 말한다. 점성은 유체가 흐름에 저항하는 정도를 나타낸다. 이 저항은 운동량, 따라서 속도의 확산을 일으키므로 세 번째 항을 확산항이라고 부른다.

외력(External Forces)

네 번째 항은 유체에 가해지는 외력에 의한 가속도를 담는다. 외력은 국소적인 힘일 수도 있고 체적력(body force)일 수도 있다. 국소적인 힘은 유체의 특정 영역에 가해지는 힘이다. 선풍기가 공기를 미는 힘이 한 예다. 중력과 같은 체적력은 유체 전체에 고르게 작용한다.

벡터 미적분을 간단히 복습한 뒤 나비에-스토크스 방정식으로 돌아오겠다. 방정식의 상세한 유도과 설명은 Chorin과 Marsden(1993), Griebel 외(1998)를 권한다.

38.2.3 벡터 미적분의 간단한 복습

원문 PDF 339쪽

식 1과 식 2에서는 기호 ∇ 가 서로 다른 세 가지 방식으로 사용된다. 이 기호는 흔히 ‘델(del)’이라고 읽으며 나블라(nabla) 연산자라고도 한다. 세 가지 사용법은 기울기(gradient), 발산(divergence), 라플라시안(Laplacian)이며 표 38-1에 정리되어 있다. 표의 아래첨자 i, j 는 데카르트 격자의 이산적인 위치를 가리킨다. δx 와 δy 는 각각 x 방향과 y 방향의 격자 간격이다. 그림 38-2를 참고하라.

Operator	Definition	Finite Difference Form
Gradient	$\nabla p = \left(\frac{\partial p}{\partial x}, \frac{\partial p}{\partial y} \right)$	$\frac{p_{i+1,j} - p_{i-1,j}}{2\delta x}, \frac{p_{i,j+1} - p_{i,j-1}}{2\delta y}$
Divergence	$\nabla \cdot \mathbf{u} = \frac{\partial u}{\partial x} + \frac{\partial v}{\partial y}$	$\frac{u_{i+1,j} - u_{i-1,j}}{2\delta x} + \frac{v_{i,j+1} - v_{i,j-1}}{2\delta y}$
Laplacian	$\nabla^2 p = \frac{\partial^2 p}{\partial x^2} + \frac{\partial^2 p}{\partial y^2}$	$\frac{p_{i+1,j} - 2p_{i,j} + p_{i-1,j}}{(\delta x)^2} + \frac{p_{i,j+1} - 2p_{i,j} + p_{i,j-1}}{(\delta y)^2}$

표 38-1. 유체 시뮬레이션에 사용되는 벡터 미적분 연산자. 열: 연산자 / 정의 / 유한차분 형태. 행: 기울기(Gradient) / 발산(Divergence) / 라플라시안(Laplacian).

스칼라장의 기울기는 그 스칼라장의 편미분들을 성분으로 갖는 벡터다. 식 2에 등장하는 발산은 중요한 물리적 의미를 갖는다. 주어진 공간 영역에서 ‘밀도’가 빠져나가는 비율이다. 나비에-스토크스 방정식에서는 흐름의 속도에 적용되며, 유체의 작은 부분을 둘러싸는 표면을 가로지르는 속도의 순변화를 측정한다. 연속 방정식(continuity equation)인 식 2는 유체의 발산이 항상 0이 되도록 하여 비압축성 가정을 강제한다. 발산 연산자의 내적은 기울기 연산자처럼 벡터를 만드는 대신 편미분들의 합을 만든다. 따라서 발산 연산자는 속도 $\mathbf{u} = (u, v)$ 와 같은 벡터장에만 적용할 수 있다.

스칼라장의 기울기는 벡터장이며, 벡터장의 발산은 스칼라장이라는 점에 주목하자. 기울기 연산의 결과에 발산 연산을 적용하면 라플라시안 연산자 $\nabla \cdot \nabla = \nabla^2$ 를 얻는다. 격자 셀이 정사각형이면, 즉 $\delta x = \delta y$ 이면 라플라시안은 다음과 같이 단순해진다. 이 장의 나머지 부분에서는 정사각형 셀을 가정한다.

$$\nabla^2 p = \frac{p_{i+1,j} + p_{i-1,j} + p_{i,j+1} + p_{i,j-1} - 4p_{i,j}}{(\delta x)^2}.$$

식 3

라플라시안은 물리학에서 흔히 등장하며, 특히 열 방정식과 같은 확산 방정식에 자주 나타난다. $\nabla^2 x = b$ 형태의 방정식을 푸아송 방정식(Poisson equation)이라고 한다. $b = 0$ 인 경우는 라플라스 방정식(Laplace's equation)이며 라플라시안이라는 이름도 여기서 유래한다. 식 2에서 라플라시안은 벡터장에 적용된다. 이는 표기를 간략하게 한 것으로, 실제로는 벡터장의 각 스칼라 성분에 연산자를 따로 적용한다.

옮긴이 주 — 마지막 문단의 ‘식 2’는 제공된 원문에 그렇게 적혀 있다. 라플라시안이 속도 벡터장에 적용되는 항은 실제로 식 1의 $\nabla \cdot \nabla \mathbf{u}$ 이며, 식 2는 발산이 0이라는 조건이다.

38.2.4 나비에-스토크스 방정식 풀기

원문 PDF 339-343쪽

나비에-스토크스 방정식의 해를 해석적으로 구할 수 있는 경우는 몇 가지 단순한 물리적 배치에 한정된다. 그러나 수치적분 기법을 사용하면 조금씩 진행하면서 해를 구할 수 있다. 우리는 시간이 흐르면서 유체의 흐름이 어떻게 변하는지를 보고 싶으므로 이러한 점진적인 수치해법이 목적에 잘 맞는다.

다른 알고리즘과 마찬가지로 나비에-스토크스 방정식을 푸는 과정도 단순한 단계들로 나누어야 한다. 여기서 사용하는 방법은 Stam(1999)의 안정 유체 기법에 기반한다. 이 절에서는 각 단계의 수학을 설명하고, 38.3절에서는 GPU에서 Cg 언어로 구현하는 방법을 설명한다.

먼저 방정식을 수치적으로 풀기 쉬운 형태로 바꾸어야 한다. 나비에-스토크스 방정식은 u, v, p 를 구할 수 있는 세 개의 방정식이라는 점을 떠올려 보자. 하지만 이를 어떻게 풀어야 할지는 곧바로 드러나지 않는다. 다음에서는 간단한 알고리즘으로 이어지는 변환을 설명한다.

헬름홀츠-호지 분해(The Helmholtz-Hodge Decomposition)

기초 벡터 미적분에 따르면 임의의 벡터 v 는 합이 v 가 되는 기저 벡터 성분들로 분해할 수 있다. 예를 들어 데카르트 격자에서는 벡터를 두 격자축 방향의 거리 쌍 $v = (x, y)$ 으로 표현하는 경우가 많다. 같은 벡터를 $v = x\hat{i} + y\hat{j}$ 라고 쓸 수도 있다. 여기서 $\hat{i}$ 와 $\hat{j}$ 는 각 격자축 방향의 단위 기저 벡터다.

벡터를 여러 벡터의 합으로 분해할 수 있는 것처럼, 벡터장도 여러 벡터장의 합으로 분해할 수 있다. 유체가 정의된 공간 영역, 여기서는 평면 영역을 D 라고 하자. 이 영역은 매끄러운, 즉 미분 가능한 경계 ∂D 를 가지며, 경계의 법선 방향은 n 이라고 하자. 그러면 Chorin과 Marsden(1993)에 제시된 다음 정리를 사용할 수 있다.

헬름홀츠-호지 분해 정리

D 위의 벡터장 w 는 다음 형태로 유일하게 분해할 수 있다.

$$w = u + \nabla p,$$

식 7

여기서 u 의 발산은 0이며 u 는 ∂D 와 평행하다. 즉 ∂D 에서 $u \cdot n = 0$ 이다.

이 정리는 증명 없이 사용한다. 자세한 설명과 증명은 Chorin과 Marsden(1993)의 1.3절을 참고하라.

이 정리는 임의의 벡터장을 두 벡터장의 합, 즉 발산이 없는 벡터장과 스칼라장의 기울기로 분해할 수 있다고 말한다. 또한 발산이 없는 장은 경계에서 0이 된다고 말한다. 이 정리는 다음 두 가지 유용한 사실을 이끌어 내는 강력한 도구다.

웁킨이 주 — 원문의 ‘경계에서 0이 된다’는 표현은 위 정리의 조건보다 강하다. 위에 명시된 조건 $u \cdot n = 0$ 은 경계에서 속도의 법선 성분이 0이라는 뜻이다. 속도 전체를 0으로 만드는 no-slip 조건은 위에서 별도로 도입한다.

첫 번째 사실

나비에-스토크스 방정식을 풀려면 매 시간 단계에서 이류, 확산, 외력 적용이라는 세 가지 계산을 수행해 속도를 갱신해야 한다. 그 결과로 얻는 새로운 속도장 w 는 발산이 0이 아니다. 그러나 연속 방정식은 각 시간 단계가 끝날 때 속도장의 발산이 0이어야 한다고 요구한다. 다행히 헬름홀츠-호지 분해 정리는 결과 압력장의 기울기를 빼면 속도의 발산을 바로잡을 수 있다고 알려준다.

$$\mathbf{u} = \mathbf{w} - \nabla p.$$

식 8

두 번째 사실

이 정리로부터 압력장을 계산하는 방법도 얻을 수 있다. 식 7의 양변에 발산 연산자를 적용하면 다음과 같다.

$$\nabla \cdot \mathbf{w} = \nabla \cdot (\mathbf{u} + \nabla p) = \nabla \cdot \mathbf{u} + \nabla^2 p.$$

식 9

그런데 식 2가 $\nabla \cdot \mathbf{u} = 0$ 을 강제하므로 식은 다음과 같이 단순해진다.

$$\nabla^2 p = \nabla \cdot \mathbf{w},$$

식 10

이는 유체의 압력에 대한 푸아송 방정식이다. 38.2.3절을 참고하라. 압력 푸아송 방정식(Poisson-pressure equation)이라고 부르기도 한다. 따라서 발산을 갖는 속도 w 를 구한 다음 식 10을 풀어 p 를 구하고, w 와 p 를 식 8에 넣어 발산이 없는 새로운 장 u 를 계산할 수 있다. 이 내용은 뒤에서 다시 다룬다.

이제 w 를 계산하는 방법이 필요하다. 이를 위해 벡터와 벡터장의 비교로 돌아가자. 내적의 정의에 따르면 벡터 r 와 단위벡터의 내적을 계산하면 r 을 그 단위벡터 위로 투영할 수 있다. 내적은 벡터 r 을 해당 단위벡터 방향의 성분으로 보내는 벡터 투영 연산자다. 헬름홀츠-호지 분해 정리를 사용하면 벡터장 w 를 발산이 없는 성분 u 로 투영하는 연산자 $\mathbb{P}$ 를 정의할 수 있다. 식 7에 $\mathbb{P}$ 를 적용하면 다음을 얻는다.

$$\mathbb{P}\mathbf{w} = \mathbb{P}\mathbf{u} + \mathbb{P}(\nabla p).$$

식 7에 투영 연산자 적용

$\mathbb{P}$ 의 정의에 따라 $\mathbb{P}\mathbf{w} = \mathbb{P}\mathbf{u} = \mathbf{u}$ 다. 따라서 $\mathbb{P}(\nabla p) = 0$ 이다. 이 생각을 이용해 나비에-스토크스 방정식을 단순화해 보자.

먼저 식 1의 양변에 투영 연산자를 적용한다.

$$\mathbb{P} \frac{\partial \mathbf{u}}{\partial t} = \mathbb{P} \left(-(\mathbf{u} \cdot \nabla) \mathbf{u} - \frac{1}{\rho} \nabla p + \nu \nabla^2 \mathbf{u} + \mathbf{F} \right).$$

식 1의 양변에 투영 연산자 적용

$\mathbf{u}$ 의 발산이 0이므로 좌변에 있는 그 도함수의 발산도 0이다. 따라서 $\mathbb{P}(\partial \mathbf{u} / \partial t) = \partial \mathbf{u} / \partial t$ 다. 또한 $\mathbb{P}(\nabla p) = 0$ 이므로 압력항이 사라지고 다음 식이 남는다.

$$\frac{\partial \mathbf{u}}{\partial t} = \mathbb{P} \left(-(\mathbf{u} \cdot \nabla) \mathbf{u} + \nu \nabla^2 \mathbf{u} + \mathbf{F} \right).$$

식 11

이 식의 좋은 점은 유체 흐름을 시뮬레이션하는 전체 알고리즘을 기호로 함축하고 있다는 것이다. 먼저 우변 괄호 안을 계산한다. 왼쪽부터 이류항, 확산항, 외력항을 계산한다. 이 세 단계를 적용하면 발산을 갖는 속도장 $\mathbf{w}$ 가 생긴다. 여기에 투영 연산자를 적용하면 발산이 없는 새로운 장 $\mathbf{u}$ 를 얻는다. 이를 위해 식 10을 풀어 p 를 구하고 식 8처럼 $\mathbf{w}$ 에서 p 의 기울기를 뺀다.

일반적인 구현에서는 식 11의 모습처럼 각 성분을 따로 계산해 모두 더하지 않는다. 대신 상태에 대한 변환들을 합성하여 해를 구한다. 다시 말해 각 성분은 하나의 장을 입력받아 새로운 장을 출력하는 단계다. 식 11을 한 시간 단계만큼 푸는 것과 동등한 연산자 $\mathbb{S}$ 를 정의할 수 있다. 이 연산자는 이류 $\mathbb{A}$, 확산 $\mathbb{D}$, 외력 적용 $\mathbb{F}$, 투영 $\mathbb{P}$ 연산자의 합성으로 정의된다.

$$\mathbb{S} = \mathbb{P} \circ \mathbb{F} \circ \mathbb{D} \circ \mathbb{A}.$$

식 12

따라서 시뮬레이션 알고리즘의 한 단계는 다음과 같이 표현된다.

$$\mathbb{S}(\mathbf{u}) = \mathbb{P} \circ \mathbb{F} \circ \mathbb{D} \circ \mathbb{A}(\mathbf{u})$$

한 시간 단계의 연산

연산자는 오른쪽에서 왼쪽으로 적용된다. 먼저 이류를 수행한 뒤 확산, 외력 적용, 투영을 차례로 수행한다. 표기를 간단히 하려고 시간은 생략했지만, 실제로는 각 연산자를 계산할 때 시간 간격을 사용해야 한다. 이제 이류와 확산 단계를 더 자세히 살펴본 다음, 푸아송 방정식의 해법으로 넘어가자.

이류

이류는 유체의 속도가 자기 자신과 유체 안의 다른 양들을 운반하는 과정이다. 어떤 양의 이류를 계산하려면 각 격자점에서 그 양을 갱신해야 한다. 속도장을 따라 양이 어떻게 이동하는지 계산하는 것이므로 각 격자 셀이 하나의 입자로 표현된다고 상상하면 이해에 도움이 된다. 이류를 계산하는 첫 시도로, 입자 시스템을 갱신하듯 격자를 갱신해 볼 수 있다. 각 입자의 위치 $\mathbf{r}$ 를 속도장 방향으로 시간 δt 동안 이동할 거리만큼 전진시키는 것이다.

$$\mathbf{r}(t + \delta t) = \mathbf{r}(t) + \mathbf{u}(t)\delta t.$$

입자 위치의 전진 적분

이를 오일러 방법(Euler's method)으로 알아볼 수도 있다. 상미분방정식을 명시적, 또는 전진 방식으로 적분하는 간단한 방법이다. 중점법이나 룽게-쿠타 방법처럼 더 정확한 방법도 있다.

이 접근에는 두 가지 문제가 있다. 첫째, 이류에 명시적 방법을 사용하는 시뮬레이션은 큰 시간 간격에서 불안정하다. $\mathbf{u}(t)\delta t$ 의 크기가 격자 셀 하나의 크기보다 커지면 계산이 '폭주'할 수 있다. 둘째는 GPU 구현에 특유한 문제다. 여기서는 fragment program으로 시뮬레이션을 구현하는데, fragment program은 자신이 기록하는 fragment의 위치를 바꿀 수 없다. 이 전진 적분 방법에는 입자를 '이동시키는' 기능이 필요하므로 당시 GPU에서는 구현할 수 없다.

해결책은 문제를 뒤집어 암시적 방법을 사용하는 것이다(Stam 1999). 현재 시간 단계 동안 입자가 어디로 이동하는지 계산하는 대신, 각 격자 셀에서 출발해 시간을 거슬러 입자의 궤적을 추적하고, 이전 위치에 있던 양을 출발 격자 셀로 복사한다. 속도, 밀도, 온도 또는 유체가 운반하는 다른 양 q 를 갱신하는 식은 다음과 같다.

$$q(\mathbf{x}, t + \delta t) = q(\mathbf{x} - \mathbf{u}(\mathbf{x}, t)\delta t, t).$$

식 13

이 방법은 GPU에서 쉽게 구현할 수 있을 뿐 아니라, Stam이 보였듯이 임의의 시간 간격과 속도에서도 안정적이다. 그림 38-3은 이중 원으로 표시된 셀에서 이류를 계산하는 과정을 보여준다. 속도장을 따라 시간을 거슬러 추적하면 초록색 x 에 도달한다. 초록색 x 에 가장 가까운 네 격자값, 그림에서 초록색 사각형으로 연결한 값들을 쌍선형 보간하고, 그 결과를 출발 격자 셀에 기록한다.

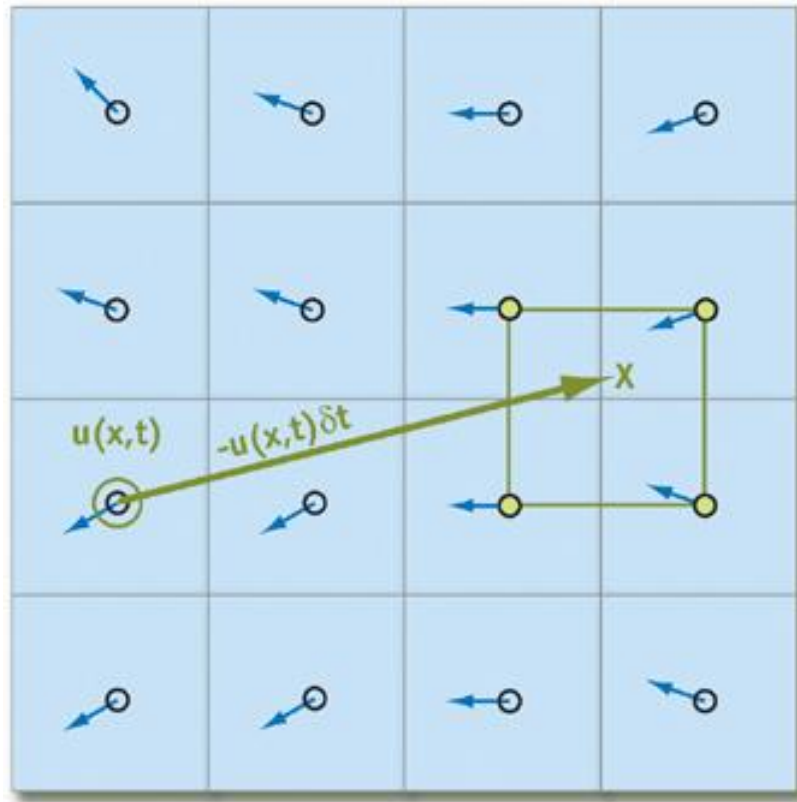


그림 38-3. 유체의 이류 계산. 도착 셀에서 이전 위치를 역추적한 뒤 인접한 네 값을 쌍선형 보간한다.

옮긴이 주 — 위의 ‘안정적’이라는 말은 큰 시간 간격에서도 수치적으로 폭주하지 않는다는 설명이다. 시간 간격을 크게 해도 정확도가 유지된다는 뜻은 아니다. 또한 fragment의 출력 위치를 바꿀 수 없다는 설명은 원문의 당시 fragment-program 구현 환경에 관한 것이다.

점성 확산(Viscous Diffusion)

앞서 설명했듯이 점성 유체는 흐름에 저항하며, 그 결과 속도가 확산되거나 소산된다. 점성 확산을 나타내는 편미분방정식은 다음과 같다.

$$\frac{\partial \mathbf{u}}{\partial t} = \nu \nabla^2 \mathbf{u}.$$

식 14

이류와 마찬가지로 이 방정식에도 여러 해법을 선택할 수 있다. 쉽게 떠올릴 수 있는 접근은 명시적인 이산 형태로 바꾸어 단순한 알고리즘을 만드는 것이다.

$$\mathbf{u}(\mathbf{x}, t + \delta t) = \mathbf{u}(\mathbf{x}, t) + \nu \delta t \nabla^2 \mathbf{u}(\mathbf{x}, t).$$

확산의 명시적 이산화

이 식의 ∇^2 는 식 3의 이산 라플라시안 연산자다. 이류를 계산하는 명시적 오일러 방법과 마찬가지로, 이 방법은 δt 와 ν 가 크면 불안정하다. 다시 Stam의 접근을 따라 식 14의 암시적 형태를 사용한다.

$$\left(\mathbf{I} - \nu \delta t \nabla^2\right) \mathbf{u}(\mathbf{x}, t + \delta t) = \mathbf{u}(\mathbf{x}, t),$$

식 15

여기서 $\mathbf{I}$ 는 단위행렬이다. 이 형태는 임의의 시간 간격과 점성에서 안정적이다. 이는 형태가 조금 감추어져 있지만 속도에 대한 푸아송 방정식이다. 헬름홀츠-호지 분해를 사용했을 때 압력에 대한 푸아송 방정식도 얻었다는 점을 기억하자. 이 방정식들은 반복 완화(iterative relaxation) 기법으로 풀 수 있다.

푸아송 방정식의 해법

우리는 압력 푸아송 방정식과 점성 확산 방정식이라는 두 푸아송 방정식을 풀어야 한다. 푸아송 방정식은 물리학에서 흔하며 잘 연구되어 있다. 여기서는 근사해에서 시작해 반복할 때마다 해를 개선하는 반복 해법을 사용한다.

푸아송 방정식은 $Ax = b$ 형태의 행렬 방정식이다. x 는 구하려는 값들의 벡터이며 여기서는 p 또는 u 다. b 는 상수 벡터, A 는 행렬이다. 우리의 경우 A 는 라플라시안 연산자 ∇^2 에 암묵적으로 표현되므로 행렬 자체를 명시적으로 저장할 필요가 없다. 반복 해법은 해의 초기 '추정값' $x^{(0)}$ 에서 시작하고, 각 단계 k 에서 개선된 해 $x^{(k)}$ 를 만든다. 위첨자는 반복 횟수를 나타낸다. 가장 단순한 반복 기법은 야코비 반복(Jacobi iteration)이라고 한다. 일반적인 행렬 방정식에 대한 야코비 반복의 유도는 Golub과 Van Loan(1996)에서 찾을 수 있다.

켈레기울기(conjugate gradient)와 다중격자(multigrid) 같은 더 정교한 방법은 더 빠르게 수렴한다. 하지만 여기서는 단순하고 구현하기 쉽다는 이유로 야코비 반복을 사용한다. 더 정교한 해법의 세부 내용과 예시는 Bolz 외(2003), Goodnight 외(2003), Krüger와 Westermann(2003)을 참고하라.

식 10과 식 15는 서로 달라 보이지만, 둘 다 식 3을 사용해 이산화하면 다음 형태로 다시 쓸 수 있다.

$$x_{i,j}^{(k+1)} = \frac{x_{i-1,j}^{(k)} + x_{i+1,j}^{(k)} + x_{i,j-1}^{(k)} + x_{i,j+1}^{(k)} + \alpha b_{i,j}}{\beta},$$

식 16

여기서 α 와 β 는 상수다. 두 방정식에서 x , b , α , β 의 값은 서로 다르다. 압력 푸아송 방정식에서는 x 가 p 를, b 가 $\nabla \cdot \mathbf{w}$ 를 나타내며, $\alpha = -(\delta x)^2$, $\beta = 4$ 다. (1) 점성 확산 방정식에서는 x 와 b 가 모두 u 를 나타내며, $\alpha = (\delta x)^2 / (\nu \delta t)$, $\beta = 4 + \alpha$ 다.

이처럼 식을 정리하면 같은 코드로 두 방정식을 모두 풀 수 있다. 해를 구하려면 각 격자 셀에 식 16을 적용하는 반복을 여러 차례 수행하면 된다. 이전 반복의 결과를 다음 반복의 입력으로 사용한다. 즉 $x^{(k+1)}$ 가 다음 반복에서 $x^{(k)}$ 가 된다. 야코비 반복은 수렴이 느리므로 많은 반복이 필요하다. 다행히 GPU에서는 각 반복의 실행 비용이 작아서 짧은 시간에 여러 번 반복할 수 있다.

옮긴이 주 — 확산에서 b 는 해당 시간 단계의 고정된 우변이고, x 는 반복하면서 갱신하는 해의 추정값이다. 둘 다 속도를 나타낸다는 원문의 설명이 반복 중 b 까지 갱신한다는 뜻은 아니다. 원문의 각주 표지 [1]은 유지했으나, 제공된 PDF의 38장에는 대응하는 각주 본문이 수록되어 있지 않다.

초기 조건과 경계 조건

유한한 영역에서 정의된 미분방정식 문제가 적절하게 정의되려면 경계 조건이 필요하다. 경계 조건은 시뮬레이션 영역의 가장자리에서 값을 어떻게 계산할지 결정한다. 또한 시간이 흐르면서 흐름이 어떻게 변하는지를 계산하려면 시작 상태, 즉 초기 조건을 알아야 한다. 이 유체 시뮬레이션에서는 처음에 모든 위치의 속도와 압력이 0이라고 가정한다. 경계 조건은 조금 더 설명이 필요하다.

매 시간 단계에서 속도와 압력이라는 두 양의 방정식을 풀므로, 둘 모두에 경계 조건이 필요하다. 직사각형 격자에서 시뮬레이션하므로 상자 안에 유체가 있고 상자의 벽을 통과해 흐를 수 없다고 가정한다. 속도에는 경계에서 속도가 0이 되는 비미끄럼(no-slip) 조건을 사용한다. 압력 푸아송 방정식의 올바른 해를 구하려면 순수 노이만(pure Neumann) 경계 조건 $\partial p / \partial n = 0$ 이 필요하다. 이는 경계에서 경계 법선 방향으로의 압력 변화율이 0이라는 뜻이다. 38.3절의 끝에서 경계 조건을 다시 살펴본다.

38.3 구현

원문 PDF 343쪽부터

문제와 기본적인 해법을 이해했으므로 이제 구현으로 넘어갈 수 있다. 알고리즘의 의사 코드를 적어 보는 것이 좋은 출발점이다. 알고리즘은 매 시간 단계에서 같으므로 다음 의사 코드는 한 시간 단계를 나타낸다. 변수 u 와 p 는 속도장과 압력장 데이터를 담는다.

```
// 식 12의 첫 세 연산자를 적용한다.
u = advect(u);
u = diffuse(u);
u = addForces(u);
// 이제 그 결과에 투영 연산자를 적용한다.
p = computePressure(u);
u = subtractPressureGradient(u, p);
```

실제로는 대부분의 연산을 제자리에서 수행할 수 없으므로 임시 저장 공간이 필요하다. 예를 들어 위 의사 코드의 이류 단계는 다음처럼 쓰는 것이 더 정확하다.

```
uTemp = advect(u);
swap(u, uTemp);
```

이 의사 코드에는 특정 구현에 종속된 내용이 없다. 같은 의사 코드로 CPU 구현과 GPU 구현을 모두 똑같이 설명할 수 있다. 우리의 목표는 모든 단계를 GPU에서 수행하는 것이다. 일부 독자에게는 이런 GPU 계산이 익숙하지 않을 수 있으므로, 일반적인 CPU 유체 시뮬레이션의 연산과 GPU에서 대응하는 연산을 비교해 보겠다.

38.3.1 CPU와 GPU의 대응 관계

모든 컴퓨터의 기초에는 메모리 모델과 처리 모델이 있다. 따라서 모든 애플리케이션은 데이터 표현과 계산을 고려해야 한다. 이 두 관점에서 CPU와 GPU의 차이를 간단히 살펴보자.

텍스처 = 배열

이 시뮬레이션은 데이터를 2차원 격자로 표현한다. CPU에서는 배열이 이러한 격자의 자연스러운 표현이다. GPU에서 배열에 대응하는 것은 텍스처다. 텍스처는 배열만큼 유연하지 않지만, 그래픽 하드웨어의 발전과 함께 그 유연성이 나아지고 있다. 당시 GPU의 텍스처는 유체 시뮬레이션을 구현하는 데 필요한 기본 연산을 모두 지원한다. 텍스처에는 보통 세 개 또는 네 개의 색상 채널이 있으므로 두 개에서 네 개의 성분을 갖는 벡터 데이터를 담기에 자연스럽다. 하나의 텍스처에 여러 스칼라장을 저장할 수도 있다. 가장 기본적인 연산인 배열, 또는 메모리 읽기는 텍스처 조회로 수행한다. 따라서 배열 오프셋에 대응하는 GPU 개념은 텍스처 좌표다. 유체의 상태를 나타내려면 최소한 속도용 텍스처 하나와 압력용 텍스처 하나가 필요하다. 흐름을 시각화하기 위해 유체가 운반하는 양을 담는 텍스처도 추가로 유지한다. 이를 ‘잉크’라고 생각하면 된다. 그림 38-4는 이 텍스처들과, 38.5.1절에서 설명할 와도(vorticity) 텍스처의 예를 보여준다.

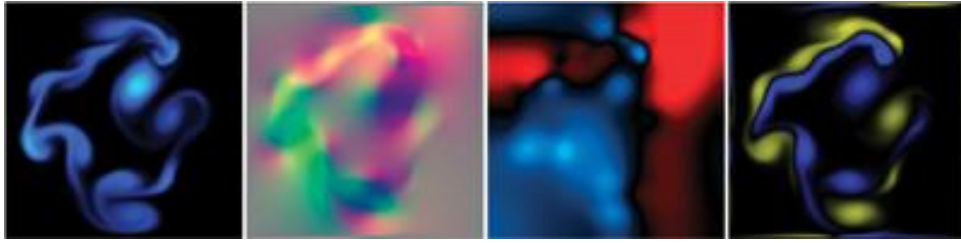


그림 38-4. 텍스처에 저장한 유체 시뮬레이션의 상태장. 본문에서 설명하는 염료·속도·압력·와도 등의 상태를 시각화한 예.

반복문의 본문 = 프래그먼트 프로그램

CPU 구현은 이중 반복문으로 격자의 각 셀을 순회하면서 알고리즘의 각 단계를 수행한다. 모든 셀에서 같은 계산을 수행한다. 당시 GPU에는 텍스처의 각 텍셀에 대해 이런 내부 반복문을 직접 수행하는 기능이 없다. 대신 프래그먼트 파이프라인은 각 프래그먼트에 같은 계산을 수행하도록 설계되어 있다. 프로그래머에게는 프래그먼트마다 프로세서가 하나씩 있고 모든 프래그먼트가 동시에 갱신되는 것처럼 보인다. 병렬 프로그래밍에서는 이를 단일 명령·다중 데이터(SIMD) 계산 모델이라고 한다. 따라서 배열을 순회하는 중첩 반복문 내부의 계산에 대응하는 GPU 연산은 각 프래그먼트에 SIMD 방식으로 적용되는 프래그먼트 프로그램이다.

피드백 = 텍스처 갱신

38.2.4절에서는 야코비 반복으로 푸아송 방정식을 푸는 방법을 설명했다. 이런 반복법은 한 번의 반복 결과를 다음 반복의 입력으로 사용한다. 이 피드백은 수치해법에서 흔히 나타난다. CPU 구현에서는 읽기와 쓰기가 모두 가능한 변수와 배열로 쉽게 구현할 수 있으므로 보통 피드백 자체를 따로 의식하지 않는다. 그러나 GPU에서는 프래그먼트 프로세서의 출력이 항상 프레임 버퍼에 기록된다. 프레임 버퍼를 직접 읽을 수 없는 2차원 배열로 생각해 보자. 프레임 버퍼의 내용을 읽을 수 있는 텍스처로 옮기는 방법은 두 가지다.

1. 텍스처로 복사(copy to texture, CTT): 프레임 버퍼에서 텍스처로 복사한다.
2. 텍스처로 렌더링(render to texture, RTT): 텍스처를 프레임 버퍼로 사용하여 GPU가 직접 그 안에 기록하게 한다.

CTT와 RTT는 기능상 같은 목적을 달성하지만 성능상 절충 관계가 있다. 일반성을 유지하기 위해 어느 하나를 사용한다고 가정하지 않고, 텍스처에 기록하는 과정을 통틀어 ‘텍스처 갱신(texture update)’이라고 부르겠다.

앞서 실제로는 알고리즘의 다섯 단계 각각이 임시 격자를 갱신한 뒤 교환을 수행한다고 설명했다. RTT에서는 읽기용으로 바인딩된 텍스처에 동시에 렌더링하면 결과가 정의되지 않으므로 피드백에 두 텍스처가 필요하다. 이때 교환은 텍스처 ID만 바꾸면 된다. 따라서 RTT의 성능 비용은 일정하다. 반면 CTT에는 텍스처 하나만 필요하다. 프레임 버퍼가 임시 격자 역할을 하고, 프레임 버퍼에서 텍스처로 데이터를 복사하여 교환을 수행한다. 이 복사의 비용은 텍스처 크기에 비례한다.

38.3.2 슬랩 연산(Slab Operations)

시뮬레이션의 각 단계를 슬랩 연산, 줄여서 slabop이라고 부르는 단위로 나눈다.〔2〕 각 slabop은 대개 프래그먼트 프로그램을 활성화한 상태에서 프레임 버퍼의 하나 이상의 프래그먼트, 흔히 모든 프래그먼트를 처리한 뒤 텍스처를 갱신하는 과정으로 이루어진다. 프래그먼트 처리는 기하 프리미티브를 렌더링해 구동한다. 이 애플리케이션에서 렌더링하는 기하는 단순하며, 사각형과 선 프리미티브만 사용한다.

각 슬랩 연산이 처리할 프래그먼트는 내부 프래그먼트와 경계 프래그먼트 두 종류다. 2차원 격자의 외곽 한 셀 너비를 경계 조건의 저장과 계산을 위해 남겨 둔다. 내부와 경계에서는 보통 서로 다른 계산을 수행한다. 내부 프래그먼트를 갱신할 때는 프레임 버퍼 가장자리의 한 픽셀 테두리를 제외한 영역 전체를 덮는 사각형을 렌더링한다. 경계 셀을 갱신할 때는 네 개의 선 프리미티브를 렌더링한다. 내부와 테두리 프래그먼트에 각각 별도의 프래그먼트 프로그램을 적용한다. 그림 38-5를 참고하라.

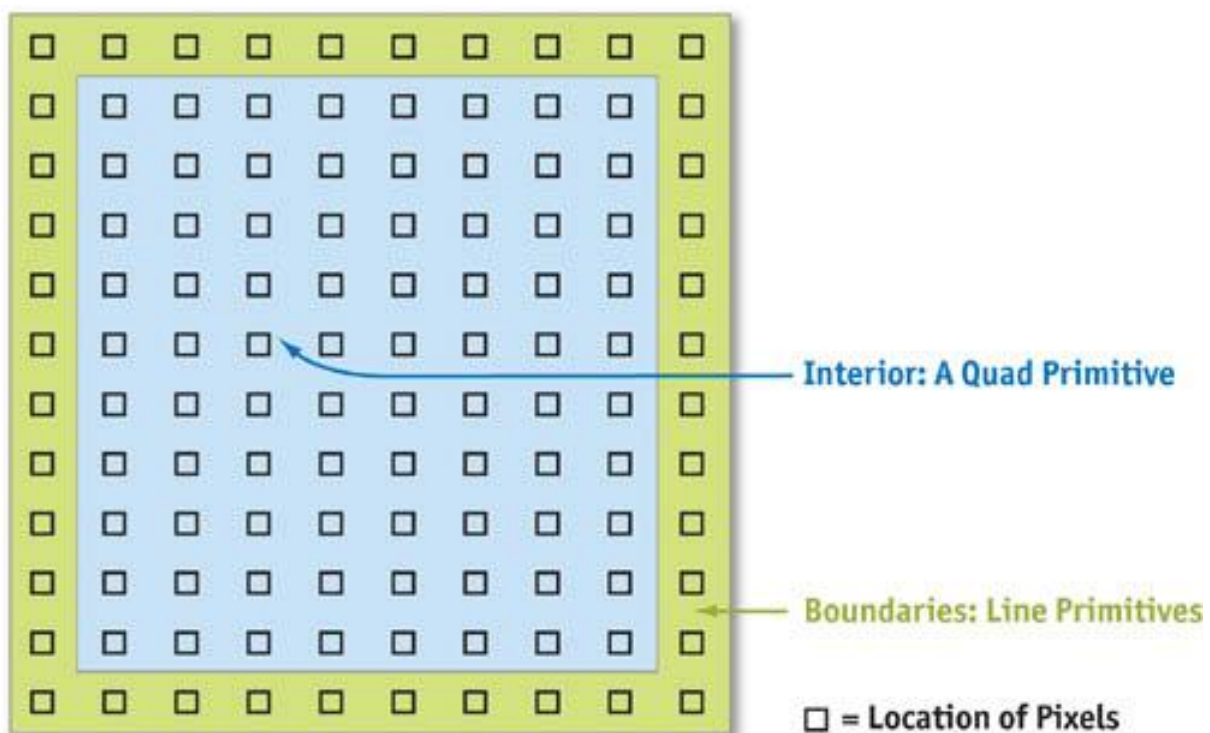


그림 38-5. 격자의 내부와 경계를 갱신하는 프리미티브. 파란 내부 영역은 사각형 프리미티브, 노란 경계 영역은 선 프리미티브로 처리한다.

옮긴이 주 — 각주 표지〔2〕는 원문대로 유지했다. 제공된 PDF의 38장에는 이 표지에 대응하는 각주 본문이 없다.

38.3.3 프래그먼트 프로그램 구현

원문 PDF 345–349쪽

알고리즘의 단계와 데이터 표현, 슬랩 연산의 수행 방법을 알았으므로 이제 각 셀의 계산을 담당하는 프래그먼트 프로그램을 작성할 수 있다.

이류

예제 38-1의 이류 프래그먼트 프로그램은 아래에 다시 적은 식 13을 거의 그대로 따른다.

$$q(\mathbf{x}, t + \delta t) = q(\mathbf{x} - \mathbf{u}(\mathbf{x}, t)\delta t, t).$$

식 13 — 다시 제시

차이가 하나 있다. 텍스처 좌표의 단위는 시뮬레이션 영역의 단위와 같지 않다. 텍스처 좌표는 $[0, N]$ 범위이며 N 은 격자 해상도다. 따라서 속도를 격자 공간의 단위로 변환해야 한다. Cg 코드에서는 해당 위치의 속도에 격자 간격 δx 의 역수인 rdx 매개변수를 곱하여 이를 반영한다. 역추적한 위치가 $[0, N]$ 범위를 벗어났을 때 경계 텍셀로 제한되도록 텍스처 래핑 모드를 CLAMP_TO_EDGE로 설정해야 한다. 뒤에서 설명할 경계 조건이 이 텍셀들을 올바르게 갱신하므로 이런 경우에도 제대로 동작한다.

예제 38-1. 이류 프래그먼트 프로그램

```
void advect(float2 coords : WPOS,    // 격자 좌표
  out float4 xNew : COLOR,          // 이류된 양
  uniform float timestep,
  uniform float rdx,                // 1 / 격자 간격
  uniform samplerRECT u,            // 입력 속도
  uniform samplerRECT x)            // 이류할 양
{
  // 속도장을 따라 시간을 거슬러 올라간다.
  float2 pos = coords - timestep * rdx * f2texRECT(u, coords);

  // 보간한 뒤 출력 프래그먼트에 기록한다.
  xNew = f4texRECTbilerp(x, pos);
}
```

이 코드에서 매개변수 u 는 속도장 텍스처이며, x 는 이류시킬 장이다. x 는 속도일 수도 있고 염료 농도 같은 다른 양일 수도 있다. 함수 `f4texRECTbilerp()`는 전달받은 텍스처 좌표에 가장 가까운 네 텍셀을 쌍선형 보간하는 유틸리티다. 당시 GPU는 부동소수점 텍스처에 대한 자동 쌍선형 보간을 지원하지 않으므로 이런 코드로 직접 구현해야 한다.

점성 확산

38.2.4절에서 설명한 야코비 반복을 바탕으로 하면 야코비 반복 프래그먼트 프로그램을 쉽게 작성할 수 있다. 예제 38-2가 그 코드다.

예제 38-2. 푸아송 방정식을 푸는 야코비 반복 프래그먼트 프로그램

```

void jacobi(half2 coords : WPOS,    // 격자 좌표
            out half4 xNew : COLOR, // 결과
            uniform half alpha,
            uniform half rBeta,      // beta의 역수
            uniform samplerRECT x,   // x 벡터 (Ax = b)
            uniform samplerRECT b)   // b 벡터 (Ax = b)
{
    // 왼쪽, 오른쪽, 아래쪽, 위쪽의 x 표본
    half4 xL = h4texRECT(x, coords - half2(1, 0));
    half4 xR = h4texRECT(x, coords + half2(1, 0));
    half4 xB = h4texRECT(x, coords - half2(0, 1));
    half4 xT = h4texRECT(x, coords + half2(0, 1));

    // 중심에서 b를 읽는다.
    half4 bC = h4texRECT(b, coords);

    // 야코비 반복식을 계산한다.
    xNew = (xL + xR + xB + xT + alpha * bC) * rBeta;
}

```

rBeta 매개변수는 식 16의 β 의 역수다. 확산 방정식을 풀 때는 alpha를 $(\delta x)^2 / (v \delta t)$ 로, rBeta를 $1 / (4 + (\delta x)^2 / (v \delta t))$ 로 설정한다. x와 b 매개변수에는 속도 텍스처를 설정한다. 그런 다음 여러 차례 반복한다. 보통 20-50회 반복하지만, 오차를 줄이기 위해 더 많이 반복할 수도 있다.

외력 적용

알고리즘에서 가장 단순한 단계는 외력에 의한 가속도를 계산하는 것이다. 동봉된 자료의 데모 애플리케이션에서는 마우스를 클릭하고 드래그하여 유체에 충격을 가할 수 있다. 이를 구현하려면 클릭한 위치의 속도 텍스처에 점 모양의 영역을 그린다. 그 영역의 색에 충격의 방향과 크기를 인코딩한다. 빨간 채널에는 x방향 크기를, 초록 채널에는 y방향 크기를 넣는다. 이 영역은 실제로 2차원 가우시안 ‘스플랫(splat)’이다.

프래그먼트 프로그램으로 각 프래그먼트가 충격 위치에서 얼마나 떨어져 있는지 확인한다. 그런 다음 색에 다음 양 c를 더한다.

$$c = F \delta t \exp \left(\frac{(x - x_p)^2 + (y - y_p)^2}{r} \right).$$

마우스 입력에 의한 외력의 스플랫 — 제공된 PDF의 수식을 그대로 보존.

여기서 F는 마우스 드래그의 방향과 길이로 계산한 힘이고, p 는 원하는 충격 반경이다. (x, y) 와 (x_p, y_p) 는 각각 창 좌표계에서의 프래그먼트 위치와 충격, 즉 클릭 위치다.

옮긴이 주 — 제공된 PDF는 본문에서 반경을 p 라고 쓰지만, 수식 이미지의 분모에는 r 을 사용한다. 또한 이미지의 지수에는 음수가 없어 거리가 멀어질수록 값이 증가한다. 이는 본문이 설명하는 국소 가우시안 스플랫과 맞지 않는다. 감쇠하는 가우시안에 음의 제곱거리 항이 필요하다. 원본 이미지는 수정하지 않았으며, 이 식을 구현에 옮길 때는 부호와 반경 매개변수의 정의를 따로 확인해야 한다.

투영 (Projection)

이 절의 앞에서 투영 단계가 두 연산으로 나뉜다는 것을 배웠다. 압력 푸아송 방정식을 풀어 p 를 구하는 연산과, 중간 속도장에서 p 의 기울기를 빼는 연산이다. 이를 위해 프래그먼트 프로그램 세 개가 필요하다. 앞서 작성한 야코비 반복 프로그램, 중간 속도장의 발산을 계산하는 프로그램, 중간 속도장에서 p 의 기울기를 빼는 프로그램이다.

예제 38-3의 발산 프로그램은 중간 속도장을 w 매개변수로 받고, 격자 간격 역수의 절반을 `half2 dx` 매개변수로 받는다. 표 38-1의 유한차분 공식에 따라 발산을 계산한다. 원문은 책의 643쪽에 있는 표를 참조한다.

발산은 임시 텍스처에 기록하며, 이를 야코비 반복 프로그램의 b 매개변수 입력으로 사용한다. 야코비 프로그램의 x 매개변수에는 압력 텍스처를 설정한다. 이 텍스처는 먼저 모든 값을 0으로 초기화한다. 즉 압력장의 초기 추정값을 0으로 잡는 것이다. α 와 $rBeta$ 는 각각 $-(\delta x)^2$ 와 $1/4$ 로 설정한다.

옮긴이 주 — 제공된 PDF에서 위 $rBeta$ 의 값은 '?'로 깨져 있다. 앞서 식 16에서 압력 방정식의 $\beta = 4$ 라고 정의했고 $rBeta$ 는 그 역수이므로, 번역문에는 $1/4$ 로 복원했다.

해를 충분히 수렴시키기 위해 보통 야코비 반복을 40~80회 수행한다. 반복 횟수를 바꾸면 시뮬레이션의 정확도가 달라진다. 오차가 눈에 띄게 되므로 20회 미만으로 줄이는 것은 좋지 않다. 더 많이 반복하면 소용돌이의 세부 표현과 전체적인 정확도가 좋아지지만 계산 시간도 늘어난다. 야코비 반복이 끝나면 압력장 텍스처를 다음 프로그램의 p 매개변수에 바인딩한다. 이 프로그램은 표 38-1의 정의에 따라 p 의 기울기를 계산하고, w 매개변수로 받은 중간 속도장 텍스처에서 그 기울기를 뺀다. 예제 38-4를 참고하라.

예제 38-3. 발산 프래그먼트 프로그램

```
void divergence(half2 coords : WPOS, // 격자 좌표
    out half4 div : COLOR,           // 발산
    uniform half half2 dx,            // 0.5 / 격자 간격
    uniform samplerRECT w)            // 벡터장
{
    half4 wL = h4texRECT(w, coords - half2(1, 0));
    half4 wR = h4texRECT(w, coords + half2(1, 0));
    half4 wB = h4texRECT(w, coords - half2(0, 1));
    half4 wT = h4texRECT(w, coords + half2(0, 1));

    div = half2 dx * ((wR.x - wL.x) + (wT.y - wB.y));
}
```

예제 38-4. 기울기 차감 프래그먼트 프로그램

```

void gradient(half2 coords : WPOS, // 격자 좌표
    out half4 uNew : COLOR,        // 새로운 속도
    uniform half halfrdx,           // 0.5 / 격자 간격
    uniform samplerRECT p,          // 압력
    uniform samplerRECT w)          // 속도
{
    half pL = h1texRECT(p, coords - half2(1, 0));
    half pR = h1texRECT(p, coords + half2(1, 0));
    half pB = h1texRECT(p, coords - half2(0, 1));
    half pT = h1texRECT(p, coords + half2(0, 1));

    uNew = h4texRECT(w, coords);
    uNew.xy -= halfrdx * half2(pR - pL, pT - pB);
}

```

경계 조건

38.2.4절에서 ‘상자 안의 유체’에는 속도가 0이 되는 비미끄럼 경계 조건과 압력에 대한 순수 노이만 경계 조건이 필요하다고 정했다. 38.3.2절에서는 격자 가장자리의 한 픽셀 테두리를 경계값 저장용으로 남겨 두면 경계 조건을 구현할 수 있다는 것을 배웠다. 값을 적절히 설정하는 프래그먼트 프로그램을 사용하여 테두리에 선 프리미티브를 그리면 이 값들을 갱신할 수 있다.

먼저 격자의 이산화가 경계 조건 계산에 어떤 영향을 주는지 살펴보자. 비미끄럼 조건은 경계에서 속도가 0이어야 한다고 요구하고, 순수 노이만 압력 조건은 경계 법선 방향의 압력 미분값이 0이어야 한다고 요구한다. 경계는 경계 셀과 가장 가까운 내부 셀 사이의 변에 놓인다고 정의하지만, 격자값은 셀 중심에서 정의한다. 따라서 각 변 양쪽에 인접한 두 셀의 평균이 경계 조건을 만족하도록 경계값을 계산해야 한다.

예를 들어 왼쪽 속도 경계에서는 다음과 같다.

$$\frac{u_{0,j} + u_{1,j}}{2} = 0, \quad \text{for } j \in [0, N],$$

식 17

여기서 N 은 격자 해상도다. 이 식을 만족하려면 $u_{0,j}$ 를 $-u_{1,j}$ 로 설정해야 한다. 압력도 비슷하게 처리할 수 있다. 도함수의 전진차분 근사를 사용하면 다음을 얻는다.

$$\frac{p_{1,j} - p_{0,j}}{\delta x} = 0.$$

식 18

이 식을 $p_{0,j}$ 에 대해 풀면 각 압력 경계값을 경계 바로 안쪽의 값과 같게 설정해야 한다는 것을 알 수 있다.

예제 38-5처럼 간단한 프래그먼트 프로그램 하나를 압력 경계와 속도 경계에 모두 사용할 수 있다.

예제 38-5. 경계 조건 프래그먼트 프로그램

```

void boundary(half2 coords : WPOS, // 격자 좌표
             half2 offset : TEX1, // 경계 오프셋
             out half4 bv : COLOR, // 출력값
             uniform half scale, // 배율 매개변수
             uniform samplerRECT x) // 상태장
{
    bv = scale * h4texRECT(x, coords + offset);
}

```

그림 38-6은 이 프로그램의 동작을 보여준다. 매개변수 x 는 내부 값을 읽어 올 속도장 또는 압력장 텍스처다. $offset$ 은 현재 경계에 인접한 내부 셀까지의 올바른 오프셋이다. $coords$ 는 현재 처리하는 프래그먼트의 텍스처 좌표 위치이므로 여기에 $offset$ 을 더하면 이웃 텍셀을 가리킨다. 각 경계에서는 바로 안쪽 텍셀을 가리키도록 $offset$ 을 설정한다. 왼쪽 경계에서는 바로 오른쪽 텍셀을 가리키도록 $(1, 0)$ 으로, 아래쪽 경계에서는 $(0, 1)$ 로 설정하는 식이다. $scale$ 은 경계에 복사할 값의 배율이다. 속도 경계에서는 -1 , 압력 경계에서는 1 로 설정하면 각각 식 17과 식 18을 올바르게 구현할 수 있다.

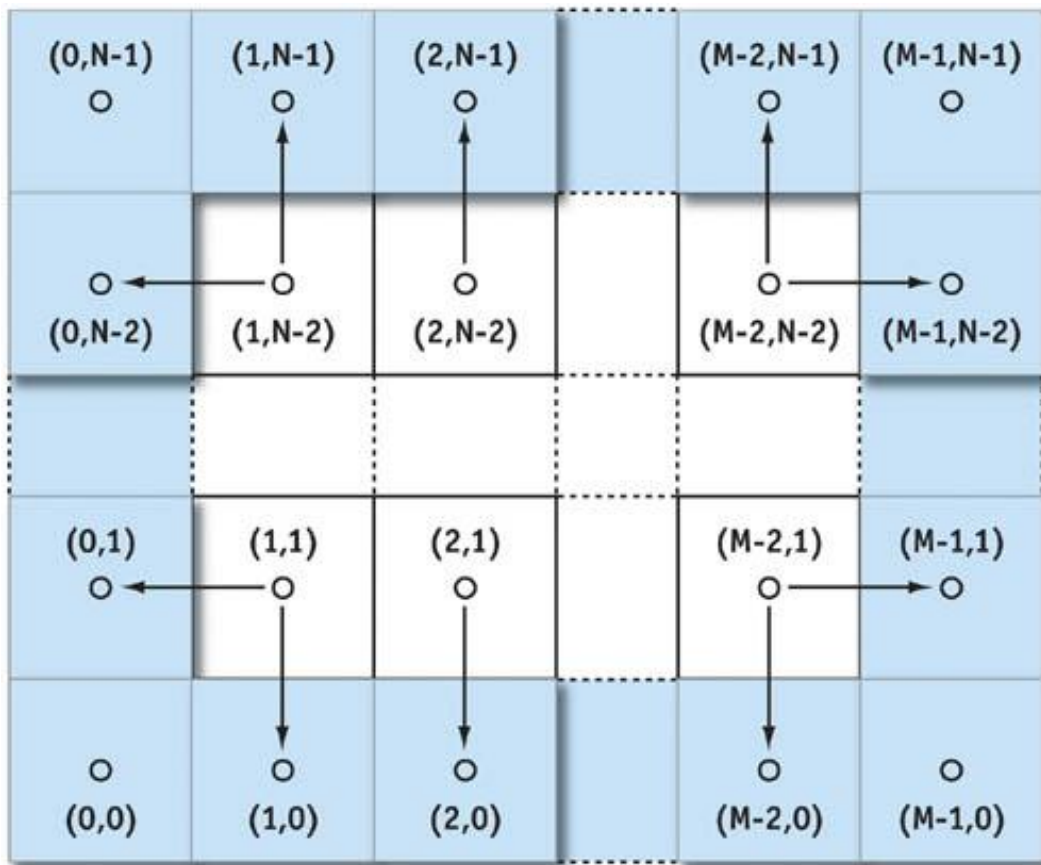


그림 38-6. $M \times N$ 격자의 경계 조건. 화살표는 내부 셀의 값을 인접한 경계 셀로 가져오는 방향을 보여준다.

38.4 응용

원문 PDF 349–350쪽

이 절에서는 이 장의 GPU 시뮬레이션 기법을 활용하는 여러 응용을 살펴본다.

38.4.1 액체와 기체 시뮬레이션

가장 직접적인 응용은 연속적인 액체 또는 기체의 체적을 시뮬레이션하는 것이다. 지금까지의 시뮬레이션은 유체의 속도만 나타내므로 그다지 흥미롭지 않다. 유체 안에 다른 것을 넣으면 더 흥미로워진다. 데모 애플리케이션은 스칼라장을 하나 더 유지하여 이를 수행한다. 이 장은 유체가 운반하는 염료의 농도를 나타낸다. RGB 텍스처이므로 실제로는 세 가지 염료 색에 하나씩 대응하는 세 스칼라장이다. 이런 양을 수동 스칼라(passive scalar)라고 한다. 유체에 실려 이동하기만 하며 유체의 흐름에는 영향을 주지 않기 때문이다.

염료의 농도를 d 라고 하면 염료장의 시간 변화는 다음 방정식의 지배를 받는다.

$$\frac{\partial d}{\partial t} = -(\mathbf{u} \cdot \nabla) d.$$

식 19

유체가 염료를 어떻게 운반하는지 시뮬레이션하려면, 속도에 했던 것과 마찬가지로 스칼라장에 이류 연산자를 적용한다. 유체 안에서 염료가 확산되는 효과까지 고려하려면 확산항을 더한다.

$$\frac{\partial d}{\partial t} = -(\mathbf{u} \cdot \nabla) d + \gamma \nabla^2 d + S,$$

식 20

여기서 γ 는 물, 또는 우리가 가정한 액체 안에서의 염료 확산 계수다. 염료 확산을 구현할 때도 속도의 점성 확산과 마찬가지로 야코비 반복을 사용한다. 다만 데모 애플리케이션은 실제로 염료의 확산을 따로 계산하지 않는다. 이류항의 수치 오차만으로도 염료가 확산되기 때문이다. 식 20에는 S 라는 항도 추가했다. 이는 염료의 공급원을 나타낸다. 애플리케이션은 클릭한 곳에 염료를 추가하여 이 항을 구현한다.

38.4.2 부력과 대류

온도는 많은 유체의 흐름에서 중요한 요인이다. 온도 변화에 따른 밀도 변화는 대류를 일으킨다. 이러한 흐름은 날씨와 바다, 호수, 심지어 커피에도 영향을 준다. 이 효과를 시뮬레이션하려면 부력을 추가해야 한다.

부력을 포함하는 가장 간단한 방법은 온도 T 를 나타내는 새 스칼라장을 추가하는 것이다. 그런 다음 해당 위치의 온도가 주어진 주변 온도 T_0 보다 높은 곳에 힘을 더하는 부력 연산자를 추가할 수 있다.

$$f_{buoy} = \sigma(T - T_0)\hat{\mathbf{j}}.$$

식 21

이 식에서 $\hat{\mathbf{j}}$ 는 수직 방향이며 σ 는 일정한 배율 계수다. 각 프래그먼트에서 식 21을 계산하고, 결과에 시간 간격을 곱한 뒤 현재 속도에 더하는 간단한 프래그먼트 프로그램으로 이 힘을 구현할 수 있다.

연기와 구름

이제 연기를 시뮬레이션하는 데 필요한 요소를 거의 모두 갖추었다. 지금까지 소개한 방법은 Fedkiw 외(2001)의 연기 시뮬레이션 기법과 비슷하다. 연기 시뮬레이션은 속도장과 압력장을 계산하는 것에 더해, 연기 밀도 d 와 온도 T 의 스칼라장도 유지해야 한다. 연기 밀도는 앞서 설명한 염료와 마찬가지로 속도장에 의해 이류된다. 밀도가 높은 연기에 작용하는 중력을 반영하도록 부력은 다음과 같이 수정된다.

$$f_{buoy} = (-\kappa d + \sigma(T - T_0))\hat{\mathbf{j}},$$

식 22

여기서 κ 는 일정한 질량 배율 계수다.

격자의 특정 위치에 연기 밀도와 온도의 공급원을 추가하면 연기를 시뮬레이션할 수 있다. 공급원은 굴뚝이나 담배 끝을 나타낼 수 있다. Fedkiw 외의 논문은 이 기본 시뮬레이션과의 차이 두 가지를 더 설명한다.

정확도를 높이기 위해 엇갈린 격자(staggered grid)를 사용하고, 연기의 소용돌이 운동을 늘리기 위해 와도 보존력(vorticity confinement force)을 더한다. 두 확장 모두 다음 절에서 설명한다.

Harris 외(2003)가 보여주었듯이 더 복잡한 시뮬레이션을 사용하면 GPU에서 구름도 시뮬레이션할 수 있다. 그림 38-7은 2차원 GPU 구름 시뮬레이션의 연속 장면이다. 구름 시뮬레이터는 유체 시뮬레이션에 부력을 포함한 열역학 시뮬레이션, 물의 응결과 증발 시뮬레이션을 결합한다. 128×128 구름 시뮬레이션은 NVIDIA GeForce FX 5950 Ultra GPU에서 초당 80회 이상의 반복으로 실행된다.

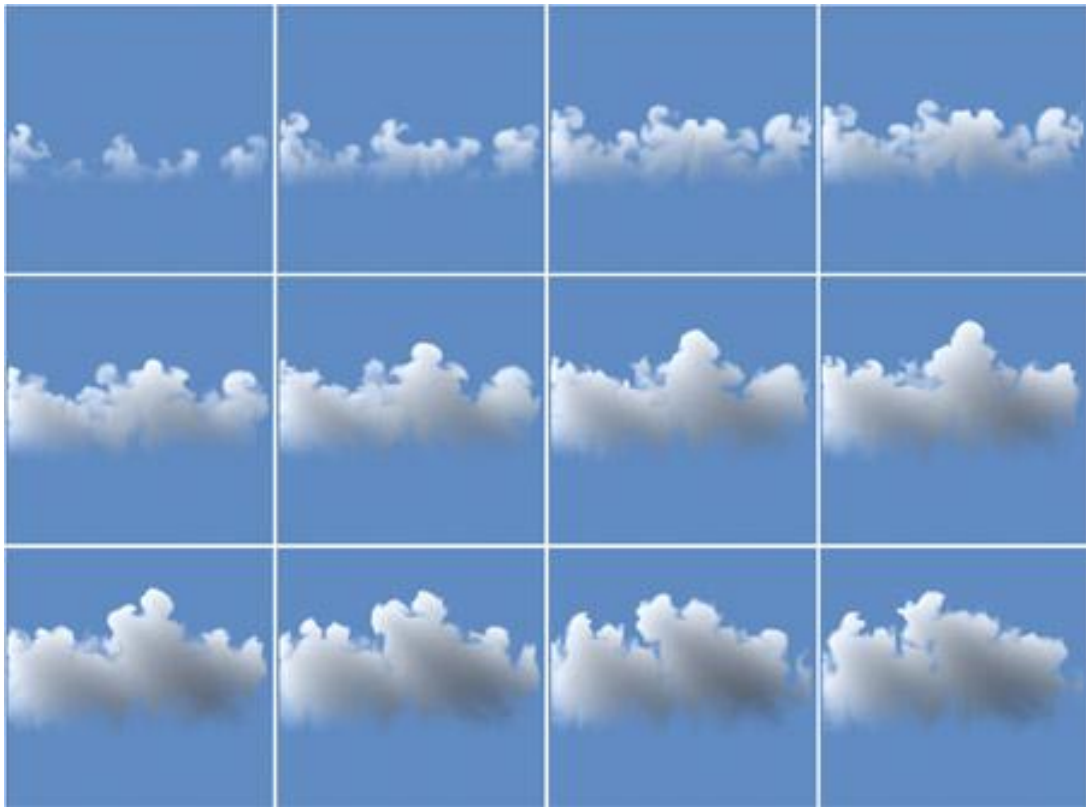


그림 38-7. 구름 시뮬레이션. 연속 장면으로 구름의 변화를 보여준다. 위 실행 속도는 원문 당시의 측정치다.

38.5 확장

원문 PDF 350–351쪽

이 장에서 소개한 유체 시뮬레이션은 더 복잡한 시뮬레이션의 기초가 되는 구성 요소다. 이 기본 시뮬레이션을 확장하는 방법은 많다. 출발점이 될 만한 몇 가지 유용한 확장을 설명하겠다.

38.5.1 와도 보존 (Vorticity Confinement)

연기, 공기, 그리고 점성이 낮은 다른 유체의 움직임에는 보통 여러 크기의 회전 흐름이 포함된다. 이러한 회전 흐름이 와도(vorticity)다. Fedkiw 외(2001)가 설명했듯이 성긴 격자 위의 시뮬레이션에서 발생하는 수치적 소산은 이런 흥미로운 특징을 약화시킨다. 따라서 이들은 작은 규모의 운동을 복원하기 위해 와도 보존 기법을 사용했다. 먼저 와도 $\omega = \nabla \times \mathbf{u}$ 를 계산한다. 와도로부터 다음과 같은 정규화된 와도 벡터장을 계산한다.

$$\Psi = \frac{\eta}{|\eta|}.$$

와도 보존에 사용하는 정규화 벡터장

여기서 다음과 같이 정의한다.

$$\eta = \nabla |\omega|.$$

와도 크기의 기울기

이 벡터장의 벡터들은 와도가 작은 영역에서 큰 영역을 향한다. 이 벡터들로부터 소산된 와도의 근사치를 복원하는 데 사용할 힘을 계산한다.

$$\mathbf{f}_{vc} = \varepsilon (\Psi \times \omega) \delta x.$$

와도 보존력

여기서 ε 는 사용자가 조절하는 배율 매개변수다. 회전(curl) 연산자 $\nabla \times$ 는 기울기와 외적 연산자의 정의로부터 유도할 수 있다. 동봉된 소스 코드에는 와도 보존이 구현되어 있다.

옮긴이 주 — 제공된 PDF에서 마지막 문단의 ε 는 본문 추출 시 빠져 있지만, 바로 위 원본 수식에서 확인할 수 있어 복원했다. ‘와도 보존’은 여기서 vorticity confinement 기법의 번역어이며, 원문 설명대로 수치적으로 소산된 회전 운동을 보강하는 방법을 뜻한다.

38.5.2 3차원

이 장에서 제시한 시뮬레이션은 모두 2차원이다. 하지만 이를 3차원으로 확장하는 데 근본적인 제약은 없다. 방정식은 본질적으로 같으며, 3차원 속도 $u = (u, v, w)$ 를 포함하도록 확장하면 된다. 프래그먼트 프로그램도 이에 맞게 다시 작성해야 한다. 2차원에서 네 이웃의 값을 읽던 부분은 3차원에서 여섯 이웃의 값을 읽도록 바뀐다.

가장 크게 바뀌는 것은 벡터장과 스칼라장의 표현 방법이다. 한 가지 선택은 3차원 텍스처를 사용하는 것이다. 하지만 3차원 부동소수점 텍스처를 지원하지 않는 하드웨어에서는 문제가 된다. 이런 경우 3차원 텍스처의 슬랩들을 2차원 텍스처 안의 격자에 타일처럼 배치할 수 있다. 예를 들어 $32 \times 32 \times 32$ 격자는 한 방향으로 여덟 타일, 다른 방향으로 네 타일을 놓아 256×128 의 2차원 텍스처에 배치할 수 있다. 평면 3차원 텍스처(flat 3D textures)라고 부르는 이 기법은 Harris 외(2003)에 상세히 설명되어 있다.

38.5.3 엇갈린 격자(Staggered Grid)

현재 시뮬레이션은 속도, 압력, 그 밖의 모든 양을 셀 중심에서 표현한다. 이는 유체를 표현하는 연속 영역을 이산화하는 한 가지 방법이며, 같은 위치 배치(collocated) 또는 셀 중심(cell-centered) 이산화라고 한다. 다른 방법은 엇갈린 격자를 사용하는 것이다. 엇갈린 격자는 압력 같은 스칼라량을 셀 중심에 두고, 속도 같은 벡터량을 셀 사이의 경계에 둔다. 구체적으로 2차원 격자에서 수평 속도 u 는 각 셀의 오른쪽 변에, 수직 속도 v 는 각 셀의 위쪽 변에 둔다. 엇갈린 격자 이산화는 여러 계산의 정확도를 높인다. 또한 셀 중심 격자에 부력 같은 힘을 적용할 때 나타날 수 있는 수치적 진동을 줄일 수 있다. 엇갈린 격자에서 유체 시뮬레이션을 구현하는 자세한 방법은 Griebel 외(1998)를 참고하라.

38.5.4 임의의 경계

지금까지 유체가 평평하고 단단한 벽을 가진 직사각형 상자 안에 있다고 가정했다. 임의의 모양과 위치를 갖는 경계가 중요하다면 시뮬레이션을 확장해야 한다.

임의의 경계를 포함하려면 38.3.3절에서 다룬 경계 조건을 임의의 위치에 적용해야 한다. 즉 각 셀에서 올바른 경계값을 계산하려면 경계가 어느 방향에 있는지 판단해야 한다. 셀마다 더 많은 판단이 필요하므로 시뮬레이션이 느려지고 복잡해진다. 하지만 장애물 주위를 흐르는 연기처럼 여러 흥미로운 효과를 만들 수 있다. Fedkiw 외(2001)처럼 움직이는 경계까지 포함할 수도 있다. 구현의 세부 내용은 해당 논문과 Griebel 외(1998)를 참고하라.

38.5.5 자유 표면 흐름

앞서 유체가 연속적이라는 가정도 했다. 우리가 표현하는 경계는 상자의 단단한 벽뿐이다. 따라서 물과 공기의 접면이 있는 바다 표면 같은 현상은 시뮬레이션할 수 없다. 이런 접면을 자유 표면(free surface)이라고 한다. 자유 표면을 포함하려면 표면이 셀들을 지나 이동할 때 그 위치를 추적해야 한다. 자유 표면 흐름의 구현 방법은 Griebel 외(1998)에서 찾을 수 있다.

38.6 결론

GPU가 제공하는 처리 능력과 프로그래밍 가능성 덕분에 다양한 현상을 빠르게 시뮬레이션할 수 있게 되었다. 그러한 현상 상당수의 바탕에는 유체 운동의 역학이 있다.

이 장을 읽었다면 GPU에서 기본적인 유체 시뮬레이션을 구현하는 데 필요한 수학과 기술을 기초 수준에서 이해했을 것이다. 이 출발점에서 자신만의 시뮬레이션 아이디어를 실험하고, 그래픽 애플리케이션에 유체 시뮬레이션을 포함할 수 있다. 이 기법들이 여러분이 활용하는 강력한 새 도구가 되기를 바란다.

38.7 참고문헌

원문 PDF 352쪽. 검색과 원문 대조를 위해 논문·도서 제목은 영문으로 유지했다.

- Bolz, J., I. Farmer, E. Grinspun, and P. Schröder. 2003. "Sparse Matrix Solvers on the GPU: Conjugate Gradients and Multigrid." *Proceedings of SIGGRAPH 2003*.
- Chorin, A. J., and J. E. Marsden. 1993. *A Mathematical Introduction to Fluid Mechanics*. 3판. Springer.
- Fedkiw, R., J. Stam, and H. W. Jensen. 2001. "Visual Simulation of Smoke." *Proceedings of SIGGRAPH 2001*.
- Golub, G. H., and C. F. Van Loan. 1996. *Matrix Computations*. 3판. The Johns Hopkins University Press.
- Goodnight, N., C. Woolley, G. Lewin, D. Luebke, and G. Humphreys. 2003. "A Multigrid Solver for Boundary Value Problems Using Programmable Graphics Hardware." *Proceedings of the SIGGRAPH/Eurographics Workshop on Graphics Hardware 2003*.
- Griebel, M., T. Dornseifer, and T. Neunhoffer. 1998. *Numerical Simulation in Fluid Dynamics: A Practical Introduction*. Society for Industrial and Applied Mathematics.
- Harris, M. J., W. V. Baxter, T. Scheuermann, and A. Lastra. 2003. "Simulation of Cloud Dynamics on Graphics Hardware." *Proceedings of the SIGGRAPH/Eurographics Workshop on Graphics Hardware 2003*.
- Krüger, J., and R. Westermann. 2003. "Linear Algebra Operators for GPU Implementation of Numerical Algorithms." *Proceedings of SIGGRAPH 2003*.
- Stam, J. 1999. "Stable Fluids." *Proceedings of SIGGRAPH 1999*.

원본 PDF 표기와 번역 편집 안내

제공된 PDF에서는 식 번호가 3에서 7로 건너뛴다. 번역본도 원본 번호를 유지했으며 식 4-6을 임의로 만들지 않았다. 본문에 끼어든 변환 잔재, 의미 없는 줄바꿈, 정리된 정리 문장 앞의 '5.-11.' 목록 번호는 읽기 좋게 정돈했다. 손실된 수학 기호는 원본 수식 이미지나 장 내부의 명시적 정의로 확인 가능한 경우에만 복원했다. 참고문헌의 분음부호와 깨진 대시 등은 정상화했다. 원문 그림과 수식의 해상도는 제공 PDF에 포함된 원본 해상도를 따른다.